CrowPanelTelemetry - Learning material

Contents

Learning material: build an application people can use **3**
Your route through the project . . . 3
Lesson 1: understand the baseline before changing it . . . 3
Lesson 2: design a UI for a real task . . . 6
Lesson 3: make the readings useful . . . 7
Lesson 4: help users understand what is happening . . . 8
Lesson 5: understand the messages without using the radio . . . 10
Lesson 6: integrate and demonstrate your application . . . 11
Student project worksheet . . . 12
Teacher notes and answer guidance . . . 15

Learning material: build an application people can use

Your task is to turn the supplied template into an application for a real classroom situation. Perhaps another student needs to check a power supply, or a visitor needs to understand an energy experiment. You choose what matters to that person, design the screen and add a useful rule based on the measurements.

You already have code for the display, touch, GPS, measurements, media, WiFi and LoRaWAN. You can use that foundation without writing your own hardware drivers. The documentation explains setup, compilation and the code. This document is where you work through exercises and keep your project record. You do not need to read it from beginning to end in one sitting.

Your route through the project

Step	What you will do	What you should keep
1. Get to know the starting program	Compile it and follow one measurement through the code	Build record and a simple sketch of the data flow
2. Design your interface	Decide who uses it, sketch screens and make a small change	Screen sketches and observations
3. Add a useful rule	Decide what a reading should mean for your experiment	A table of examples and expected results
4. Keep buttons and feedback clear	Follow work that takes time, such as playing a file	A timeline and normal/error checks
5. Understand the messages	Decode sample data and explore radio limits on your computer	Calculations and an explanation
6. Put your application together	Demonstrate it and explain your choices	A small, reproducible submission

For the practical session, use the wiring activity together with the connection tables in the documentation.

Use the project worksheet as you go. The teacher notes and answer guidance are at the end. They are a teaching resource, rather than extra setup you must complete.

You should be comfortable with basic C++ functions, variables and structs. If a term is unfamiliar, use the glossary in the documentation or ask your teacher. It is fine to start with the offline exercises while hardware is being prepared. Keep radio credentials blank during ordinary interface work, and use the teacher's approved low-voltage setup for practical experiments.

Lesson 1: understand the baseline before changing it

Question

Imagine someone taps Next image while the panel is connecting to the network. Why should they still be able to use the screen? Find how this program lets several jobs make progress without making every button wait for them.

Before you run anything

Read how the program shares the work with `App.h` and `CrowPanelTelemetry.ino` open. Write down your predictions for these cases:

1. The sketch boots with blank radio credentials and no SD card.
2. A valid 0 A reading arrives.
3. The sensor cannot be calibrated.
4. A demo button is tapped while the radio is waiting for airtime.
5. The program misses two measurement deadlines.

For each case, ask: Which part of the program handles this? What changes? What should appear on the screen? Revisit your predictions after following the code.

Activity A: keep a record of your first build

Work through Compile the template. Choose Arduino IDE, Arduino CLI or PlatformIO, complete the common preparation and your route's build procedure, then finish the guide's baseline checklist. The three routes use the same source and patched dependencies; do not create another firmware project.

Record the source revision, core/library versions, successful patch checks, selected dependency paths, actual P4 Chip Variant, partition, command or IDE settings, and flash/RAM summary in worksheet section 2. Keep the complete build output so another student can reproduce your result.

Compile with the delivered blank radio credentials. Host checks, if their tools are available, provide additional evidence; they do not emulate the panel. Record missing prerequisites and unexecuted hardware checks explicitly. The setup reference explains configuration details when you need to look something up.

Practical activity: follow the wires before you switch on

Use the panel and module wiring section in the documentation. This activity is about understanding the connections, not seeing who can plug everything in fastest. You can complete the drawing and questions before the hardware is ready.

Work in pairs, with power disconnected. One person reads the table and the other finds the contact and follows the wire. Swap roles for the next module.

Module	Find on the panel	Explain to your partner
INA226 measurement board	J13: SCL, SDA, 3V3 and GND	Why do the signal wires and supply use the 3.3 V level? Which separate terminals carry the measured load current?
GT-U7 GPS	J2.1 RX and a common ground	Why does the GPS TXD wire go to RX? Which supply and serial-level facts are still needed for this exact breakout?
Core1262 radio	J7: SPI, control signals, 3V3 and ground	What is the difference between J7.20 and GPIO20? Where does DIO1 go, and why is DIO2 left open in this wiring?

On a copy of the wiring picture, mark one complete signal path for each module. Then mark the ground and power path in different colors. Leave the GPS power path marked **not yet connected** until the teacher confirms its rating.

Before any powered work, explain these four points:

- A connector contact number and a GPIO number are different labels.
- The INA226 has both electronics-power pins and a separate measurement path.
- The GPS can send valid serial messages before it has a position fix.
- The LoRa radio uses an antenna and gateway; panel WiFi is a different connection.

Record the checks in worksheet section 3. Ask the teacher to check the assembly before applying power. You do not need to transmit a radio message to finish this wiring activity. After a supervised power-on, compare only the functions you actually tested with your predictions; keep the remaining items marked as not checked.

Activity B: follow one measurement

Start at `nextSample` in `Application.cpp`. Follow these steps in the source:

- Determine when the first slot is due.
- Find where `Reading` starts as unavailable and becomes a real conversion.
- Find how it enters `collecting.samples` and then the batch queue.
- Locate the worker's fragment offset and payload-size calculation.
- Find the encoder and matching JavaScript decoder.
- Locate the display's finite-value check.

Draw a small diagram with these owners and values. Add an invalid measurement path to the same diagram. Explain why an invalid slot still has an uptime minute.

Activity C: follow one button press

Follow the Demo hit rectangle, `tap()`, `demoRequests`, the radio service block and the later radio event. Identify the exact point at which queueing is accepted and the different point at which a transmission result is reported.

What happens if the same button is tapped repeatedly? Why would a spinner labelled "sent" immediately on tap be misleading?

Checkpoint

You can explain, in your own words:

- The distinction between the application loop and worker tasks.
- Why `Config.h` is the place to check pins.
- Why a missing value must not be displayed or encoded as a measured zero.
- Why ten-minute batching is different from guaranteed ten-minute delivery.
- Which setup checks require actual hardware.

Submit your baseline record, the measurement diagram and corrected predictions. Keep a short list of uncertainties instead of guessing missing hardware facts.

Optional investigation

Compare a missed slot with a fresh sensor failure. Both can be invalid on air, but they have different causes. Does the existing UI preserve enough information for your proposed application? Write a requirement before proposing new state.

Lesson 2: design a UI for a real task

Goal

Create a screen flow that helps your chosen user make a decision. You may redesign the template's appearance. Keep its hardware/data interfaces and error meaning.

Activity A: define the user and make a wireframe

Choose a low-voltage classroom application: a DC supply monitor, an experiment status panel, a small energy-demonstrator dashboard or another instructor-approved use. State who uses it, what they need to notice first and which action follows.

Sketch at 1024×600. Include a screen map showing navigation, a place for unavailable measurements, local/network status and a visible way to reach configuration. Decide whether all information belongs on the first screen. A student conducting an experiment and a visitor observing it may need different priorities.

Mark each control's rectangle on the sketch. Check that the visual button and its hit target occupy the same region. Specify what a held finger should do.

Activity B: integrate one small change first

Use the worked voltage-caption example. Read why it uses the existing model and why NaN is checked before the threshold. Compile it and review the diff. Do not add a second INA226 read to the renderer.

Use this table as your expected result for the example 4.8 V threshold:

Input	Expected caption
NaN	VOLTAGE / NO DATA
0 V	VOLTAGE / LOW
4.79 V	VOLTAGE / LOW
4.8 V	VOLTAGE
5.1 V	VOLTAGE

The table is reasoning with example test values. Do not alter production measurements to fake these values on an operational board. Use a clearly separated host exercise or an instructor-controlled test input when physically checking boundary cases.

Activity C: implement your layout

Work in `Ui.cpp`, following the extension reference. Move drawing and hit testing together. Reuse the existing model/status information and commands. Let the existing background jobs handle slow operations.

Test these flows in your design:

- Open a secondary screen, then return to the dashboard.
- Receive a changed status while a secondary screen remains open.
- Change a displayed value from long text to short text.
- Tap near every edge of a button and outside it.
- Hold a finger down instead of tapping.

- Encounter unavailable sensor, media or network data.

Explain which elements redraw once on entry and which redraw periodically. If redrawing is noticeably slow, identify what is being drawn or blocked before adding another task.

Checkpoint

Submit the user/task brief, wireframe, screen map, UI diff and an observation table. Label a design preview as a mockup; label actual board screenshots as hardware captures. Include one unavailable-data example and one queue-full/pending example.

Reflection

Which information did you emphasize or move? What evidence would show that your user can find it faster? Describe a simple peer test with a task, an observable outcome and a note-taking method. Visual polish alone does not establish usability.

Lesson 3: make the readings useful

Goal

Add one useful application rule. State exactly what it means and handle unavailable input. Calculate the decision in one place so every screen gives the same answer.

Activity A: choose a rule and its boundary

Examples include “supply below the experiment’s lower limit,” “current outside the expected operating band,” or “the latest measurement cannot support a decision.” Choose a meaning justified by the actual experiment and calibrated hardware. An arbitrary threshold copied from another voltage system is not a requirement.

Write a table of inputs and expected output. Include the exact threshold, nearby values, a real zero, NaN, calibration unavailable and a missed sample slot. Use physical units in the table.

Decide whether the rule needs memory:

- A caption describing the current reading can be derived.
- A warning that remains until acknowledged must remember something.
- A minimum since boot is an accumulator, updated once per valid sample.
- Hysteresis requires remembering which side of a band the application is on.

Choose just one of these ideas to start with. A small rule that you can explain and check is a better starting point than several features added at once.

Activity B: place the rule correctly

For a presentation-only caption, use the model as demonstrated in lesson 2. If several screens or actions need the same decision, calculate it in `Application.cpp` and expose its result through the model/interface. The UI then presents the result without recomputing a competing rule.

If you process each new sample, identify the exact point where a new reading is taken. The screen refreshes about once a second while measurements occur once a minute; counting

refreshes would process the same reading repeatedly. Missed old slots must not update a historic minimum as if a real zero had been observed.

Before adding a variable, write down what it remembers, where it belongs, when it changes and what happens after a restart. You probably do not need another background job for a simple rule. Start by seeing what the existing loop can do.

Activity C: specify a stateful extension, if required

For an optional warning that stays on until it is cleared, define transitions such as:

Current condition	New event	Question your requirements must answer
Normal	Valid out-of-range sample	Does the warning latch immediately?
Warning active	Valid in-range sample	Clear automatically or keep it latched?
Warning active	User acknowledges	Clear now, or wait for a healthy measurement?
Any	Invalid sample	Show uncertainty, retain a historical warning, or both?
Any	Reboot	Recompute or restore state?

Complete the table before implementation. These are design questions, not existing template behavior. An acknowledgement handled within the same application/UI context may use a short application function; it does not inherently require a new FreeRTOS task.

Checkpoint

Submit the rule specification, ownership explanation and boundary test results. Show how the interface distinguishes “normal,” “warning,” and “cannot decide.” Explain whether the rule changes only presentation, application state, or the wire protocol. If it does not need new telemetry, leave the existing encoder unchanged.

Optional challenge

Define a two-threshold hysteresis rule and a short sequence that would cause a single-threshold warning to flicker. Predict every transition. Your state machine should have a clear owner and one decision table, not a separate set of flags in each screen.

Lesson 4: help users understand what is happening

Goal

Integrate image, sound and connection actions without blocking the UI. Make the user-visible distinction between requested, queued, completed and failed work.

Activity A: draw an action timeline

Choose Play, Next image or Connect. Trace the actual code from `tap()` to the worker and the later event. Draw a timeline with UI, queue, worker and application lanes. Mark where the request is copied and whether it contains a value or an allocation pointer.

For Play, identify who reads the file, who owns PCM after queueing, and who frees it on queue failure versus playback completion. Explain why JPEG browsing does not read from the SD inside the audio task.

Activity B: define visible states

For each action you expose in your design, complete this worksheet:

Stage	What your user sees	Evidence that allows that statement
Request rejected	A specific reason	Input validation or queue result
Request accepted	Queued/pending indication	Successful command send
Work in progress	Current worker status	Worker event
Work completed	Result	Completion event or accepted image
Work failed	Recoverable action or restart guidance	Failure event and documented policy

A button press does not prove playback, connection or radio delivery. Decide where the user sees feedback if they navigate away before work finishes.

Activity C: prepare and inspect resources

Read the image and sound-file requirements. Use two teacher-checked baseline JPEGs and one short WAV in the required simple format. Predict filename order and wrap behavior. Calculate the PCM duration from byte count before playing it.

With approved classroom hardware, try a missing image folder and a missing audio file. Check that your screen remains usable and reports the error. Do not perform arbitrary malformed-file fuzzing or repeated SD/Hosted stress on the classroom baseline; those are separate engineering experiments with known limits.

Activity D: reason about queue capacity

The media-command queue holds four requests. The sound queue holds one waiting buffer in addition to a currently playing one. A one-item demo queue holds one pending demo. The WiFi command queue replaces a pending request with the latest.

Which behavior is appropriate for your interface: reject another action, show busy, or intentionally replace a pending choice? Explain it with the existing contracts. A larger queue is not automatically better: it can make the user wait for a long sequence they no longer want.

Checkpoint

Submit the timeline, visible-state worksheet and a test record for normal and failure paths. Show that your renderer and touch handler do not perform file reads, wait for WiFi, call LoRaWAN joins or stream PCM directly.

Lesson 5: understand the messages without using the radio

Goal

Decode the template's actual wire format, preserve sample meaning and explain why radio delivery can be slower than acquisition. Most of this lesson runs offline.

Activity A: inspect the reference packet

Run the offline Node decoder example. Open `tests/reference-packet.json`, `Telemetry.h` and `ttn-decoder.js` side by side. Answer these questions using the actual bytes:

1. What are the version, boot ID and first sample minute?
2. How many samples are present and what is their first offset?
3. Which sample indices are invalid, and which valid sample contains zero?
4. What are the decoded values of the first sample?
5. Which part of the result depends on reception time?

Explain why the file contains deliberate invalid test examples. It is a test vector, not a ten-minute capture from a physical device.

Activity B: optional closer look at the bytes

Your teacher can choose this activity if the course includes binary data. You can use the template without changing its message format.

A sample has 11 voltage bits, signed 11 current bits and 10 power bits. Explain how those add to four bytes. Find the scale factors and rounding in the encoder.

Calculate the quantized integers for 12.34 V, −0.37 A and 4.6 W. Explain how a negative current is represented in the 11-bit field and recovered by the decoder. Then explain why FFFFFFFF differs from valid `00000000`.

Check the distinction between the wire range and measurement range checked for your hardware. A packet field that can represent 10.23 A does not mean your shunt/module can carry it.

Activity C: fragment a batch

Derive the full packet size from the seven-byte header and four-byte samples. Now assume only 36 application bytes are available because of pending MAC commands. How many whole samples fit? What are the two fragment sizes and descriptor bytes? Which fields let a receiver combine the fragments without using arrival order?

Use `tests/codec.cpp` to check your reasoning. Do not remove invalid sample slots to make a packet smaller: that would change the relationship between time and index.

Activity D: calculate the delivery constraint

Run:

```
python3 tools/airtime.py
```

Read its assumptions and the airtime reference. For SF7 and SF12, compare one full packet's predicted airtime and 144 ten-minute batches per day. Explain what happens to the queue when acquisition is faster than delivery. Include joins, demos, pending MAC commands and fragmentation in your explanation of why the simple full-packet table is not a complete network budget.

Do not shorten the physical sample/transmit loop for this exercise. Host analysis provides fast feedback without consuming public-network airtime.

Activity E: predict a Class-A downlink

Draw an uplink followed by RX1 and RX2. A message is queued on the network server between two permitted uplinks. When could it reach the panel? What would the user see before then? Why does successful panel WiFi not answer either question?

A supervised RF exercise may demonstrate one demo and one downlink using the instructor's assigned registration and test plan. It is optional for completing the offline packet reasoning.

Checkpoint

Submit the decoded test example, calculations, fragment worksheet and delivery timeline. Distinguish acquisition time, queue acceptance, local TX completion and network receipt. Explain why the decoder alone does not maintain a whole device history.

Lesson 6: integrate and demonstrate your application

Project brief

Create an application for an instructor-approved low-voltage classroom scenario. Use this template's established peripherals and transports, design your own UI/UX, and add at least one useful application rule that handles missing readings. Keep the code small enough that a teammate can explain its owners and data path.

Possible scenarios include an experiment supply monitor, a portable measurement station or an energy-demonstrator interface. You must define the intended user and conditions under which it will be used; the scenario name alone does not specify behavior.

Required behavior

Your interface must present meaningful measurements and units, distinguish missing input from a valid zero, expose GPS freshness and show media/network status. The user must be able to navigate your screens, configure WiFi and access the existing image/audio/demo/downlink functions through a coherent flow.

Your application rule must have a written input/output or transition table. It must process data at the correct sampling boundary when appropriate. Calculate the decision in one place and demonstrate invalid-input and recovery behavior.

Preserve the baseline pin mapping, sample-slot semantics, bounded queues, nonce handling and radio pacing. A deliberate message-format or peripheral change is an extension

requiring a separate instructor-approved scope, matching encoder/decoder changes and appropriate tests. It is not necessary to complete this project.

Integration sequence

1. Review your application brief and UI screen map with a peer.
2. Build the baseline and record the toolchain/configuration.
3. Implement the interface using the existing model and command interfaces.
4. Add the application rule and test its boundaries.
5. Verify media/configuration interaction and status reporting.
6. Run relevant host checks, formatting and the selected firmware compile.
7. Perform only the physical checks approved for your supplied kit.
8. Demonstrate normal operation and at least one understandable failure case.

Keep changes coherent. If an iteration fails, preserve the observation and explain what boundary failed before adding another feature.

Submission package

Use the project worksheet. Include:

- A short application/user brief and the final screen map.
- The source diff and an example configuration without credentials.
- A diagram identifying model, UI, worker and persistence ownership.
- Boundary/transition tables for your application rule.
- Build settings, dependency versions and selected paths.
- Host/compile results and any actual hardware observations, labelled separately.
- A short demo plan with success, unavailable data and queued/failed work.
- Known limitations, including any unresolved physical prerequisite.

Do not claim radio delivery from a local TX message or hardware operation from a compile. If classroom hardware is unavailable, submit the corresponding offline and compile evidence and state the missing physical checks explicitly.

Oral explanation

Be ready to answer these questions while showing the source:

- Where is a new sample created, and how do you avoid processing it twice?
- What happens if your rule receives NaN or a legitimate zero?
- Who owns an image or audio allocation after a successful queue send?
- What does your interface promise when a command is queued?
- What happens to samples during a prolonged network outage?
- Which change would require touching both encoder and decoder?
- What would you investigate first if SD activity reset the panel?

A clear explanation of a limitation is stronger evidence than an unsupported claim that the entire system is production-ready.

Student project worksheet

Copy this worksheet into your course submission location and complete it as you work. Keep credentials out of it. Use “not checked” when evidence is unavailable; do not turn an assumption into a measured result.

1. Application and user

- Project title:
- Team and responsibilities:
- Intended user and task:
- Low-voltage experiment/system being observed:
- Decision supported by the interface:
- Actions the application actually performs:
- Functions outside your project scope:

2. Record your starting build

Complete the student compilation guide with one selected route. Save the complete log alongside this worksheet; a screenshot of the last successful line alone cannot establish which dependencies were used.

Item	Your recorded value/evidence
Starting source revision	
Working branch/revision used for submission	
Arduino CLI/IDE or PlatformIO Core/platform version	
esp32 core and external library versions	
RadioLib and JPEGDEC patch verification	
Selected dependency paths	
Board model/revision and actual P4 silicon	
Selected route and full command, or IDE Tools settings	
FQBN/PlatformIO environment, CPU, flash, PSRAM and partition	
Baseline flash/static-RAM usage and complete log location	
C6 firmware evidence, if physically checked	
Baseline compile/host check results	

3. Hardware assumptions and checks

Item	Verified value	Evidence or remaining uncertainty
Supply/cable and module rail headroom		
Connector orientation and pin mapping		
GPS VCC/TXD levels and baud		
INA226 shunt/rating/address/pull-ups		
Test load/range and calibration limit		
Radio TCXO population and switch arrangement		
Antenna/power/RF test approval		
Display/touch/audio observations		

Reference the instructor's kit record where appropriate. Do not silently repeat an unverified value from a photograph.

4. UI/UX

Attach a screen map and wireframe. For each screen, identify the user goal, information shown, controls, pending/error feedback and return path. Record which rectangles are used

for drawing and touch hit testing. Explain missing/stale data and at least one long-to-short text redraw case.

User task	Expected interaction	Observation/peer feedback	Change made

5. Application rule

- Rule in plain language:
- Input units and acceptable range:
- Source of each threshold:
- Derived result or remembered state:
- Authoritative owner and initialization:
- Exact update event, including sample freshness:
- Missing-data and recovery behavior:
- Reboot/persistence behavior:

Input/state/event	Expected result	Test method	Actual result
Missing input / NaN			
Real zero			
Below boundary			
Exact boundary			
Above boundary			
Recovery or acknowledgement			

6. What happens while work is still running

Choose at least one request and draw its complete command/event timeline. Record producer, queue/item capacity, consumer, immediate feedback, completion and failure. For an allocation, identify who frees it on both successful and failed queue sends.

7. Explain the data being sent

- Version/FPorts used:
- Whether the wire format changed; coordinated changes if so:
- Full packet and fragment sizes:
- Acquisition interval versus actual delivery expectation:
- Queue capacity, overflow behavior and reboot loss:
- Decoder/reassembly assumptions:
- Airtime calculation and RF evidence category:

8. Checks and submission

Check	Exact command or procedure	Configuration/input	Result and evidence
Formatting			
Changed rule/interface			
Relevant host tests			

Check	Exact command or procedure	Configuration/input	Result and evidence
Firmware compile			
UI interaction			
Approved hardware checks			

List files changed, known limitations and unexecuted checks. Separate calculations, host simulations, compile results and measured hardware observations. End with instructions that let another student reproduce your selected build and understand your application without needing a live verbal explanation.

Teacher notes and answer guidance

Purpose and teaching boundaries

The aim is for students to build a useful application using the hardware code already supplied. They should decide who the application is for, design the screens and explain what the readings mean. Along the way, they learn why the screen should not have to wait while a file is read or a network connects.

This is not a course requirement to repair every documented upstream limitation. Scope extensions separately. Keep the distinction between a controlled classroom template and unattended public-network operation visible throughout assessment.

The repository has compile/host evidence, but its recorded physical acceptance is incomplete. Before a practical session, establish the actual kit's wiring, voltage, shunt, oscillator, antenna/power, C6 firmware and display/touch/audio behavior. Record that evidence rather than assuming the students' kit matches a reviewed schematic population.

Prepare the classroom baseline

1. Choose and record the firmware revision and exact dependency versions.
2. Provide the complete sketch, tool prerequisites and a clear dependency location.
3. Verify both patches and selected library paths; stock JPEGDEC can compile.
4. Prepare known-good baseline JPEGs and short WAV files in the required format with suitable licenses.
5. Establish which exercises are host-only and which have approved physical checks.
6. Assign radio registrations and a public/private network test plan when needed.
7. Supply a calibrated, current-limited low-voltage experiment and shunt information.
8. Tell students what hardware behavior has and has not been verified.

A shared instructor demonstration can cover a slow or airtime-limited network step. Students can still complete packet, queue and UI reasoning offline. Do not solve a class-time problem by making every board transmit frequently on public TTN. Coordinate RF sessions so that demo traffic and downlinks fit the applicable plan.

Use the compilation instructions before lesson 1. Select a primary classroom route while allowing the alternative tools against the same dependency collection. Provide the actual silicon identification, exact release sources and a known-good build log. On managed machines, prepare packages and Python/patch prerequisites before the session; allow download time and disk space for the first PlatformIO build. Check that VS Code inherits DEPS and uses Core 6.2.0 if that route is offered.

Assess reproducibility through worksheet section 2: selected paths, patch checks, configuration, full log and memory summary. Students need one successful route, not three installs. Keep GUI guidance distinct from the recorded CLI validation; no GUI or physical-operation result is implied by the instructions.

Suggested teaching rhythm

Use each lesson as a cycle: predict, inspect, change, test and explain. Begin with a short prediction rather than a code walkthrough. After a checkpoint, ask a peer to follow the student's diagram or perform a task on the UI.

Students can work individually or divide UI, application and validation work in a team. Every member should still follow a complete measurement and button action. Avoid teams where only one person understands the queue or decoder boundary.

For an abbreviated course, require the baseline, UI, one derived application rule and offline telemetry lesson. Stateful alarms, new application downlink contracts and protocol changes are optional extensions with additional review/testing cost. Do not require a new framework or extra worker merely to make a project look larger.

Checkpoint guidance and answer notes

Lesson 1: baseline and ownership Expected source trace: `loopApplication` schedules a slot, obtains a coherent conversion when available, puts it into `collecting`, and sends a complete Batch to `batches` after ten slots. Radio service selects whole samples using the current payload allowance and calls the encoder; the JavaScript decoder interprets the same fields. UI rendering reads the model, not the sensor directly.

A blank radio configuration produces a visible configuration message and prevents RF. A missing SD produces a media status; it does not by itself justify inventing measurements. A legitimate zero is finite. Failed calibration leaves invalid triplets. Missed old slots are represented as invalid slots with their original indices. Queueing a demo is not transmission or gateway receipt.

Watch for diagrams that connect the touch handler directly to every device. Ask who owns a value after the initiating function has returned.

Lesson 2: UI/UX The worked 4.8 V caption table is: NaN → NO DATA; 0 and 4.79 → LOW; 4.8 and 5.1 → ordinary VOLTAGE. The exact equality behavior follows <, not <=. Ask why the threshold is appropriate for the assigned experiment.

Assess matched draw/hit rectangles, screen-entry redraw and status changes while a screen stays open. Students should retain a way to see calibration/errors/drops in their own layout. They may redesign visual appearance; preserving the stock layout is not the assignment.

Lesson 3: application meaning A current-value caption can be derived. A warning that stays on until it is cleared, minimum since boot or hysteresis needs memory. That memory belongs to the application, with documented initialization and transition rules. Refresh-based sample counting and independent threshold decisions in multiple consumers indicate an ownership problem.

Accept different acknowledgement policies if they are specified and tested. Do not supply an implicit answer to “what happens when the sensor becomes unavailable?” The student must separate an unknown current condition from any remembered event.

Lesson 4: asynchronous work Expected Play path: touch → media command → file validation/loading → sound queue → audio writes → media completion/error event → application model → UI. The media worker frees a sound buffer if queueing fails; the audio worker frees it after a successful transfer. The image path has a separate ownership transfer through `presentImage`, which nulls the event pointer after adoption.

Ask why an event queue that can block producers makes a long UI-loop stall matter. A student who only increases capacities has not necessarily identified the slow consumer or defined sensible pending-action behavior.

Lesson 5: packets and airtime The reference packet has version 1, boot ID `0x1234` (4660), first sample minute `0x010203` (66051), offset 0 and ten samples. Sample indices 3–7 are invalid; index 8 is a valid all-zero reading. The first sample decodes to 12.34 V, −0.37 A and 4.6 W. Those fields contain uptime/sample information; reception time does not create it.

For the first sample, quantized integers are 617, −37 and 46. The signed 11-bit current field stores 2011 (`0x7DB`), and sign extension recovers −37. The packed bytes are `4D 3F 6C 2E`. Four bytes hold 11+11+10 bits. The full packet has 47 application bytes. At 36 bytes allowance, seven samples fit in 35 bytes; three remain for 19 bytes. Descriptors are `0x07` and `0x73`.

At SF7, the reference full packet is 112.896 ms; 144 batches use about 16.257 s before joins/MAC/demo extras. At SF12 it is 2629.632 ms; 144 batches use about 378.667 s. Acquisition can remain every minute while delivery falls behind and eventually drops new batches from the bounded queue. The stateless decoder does not reassemble or persist an entire history.

Lesson 6: integration Require a traceable relationship between the application brief, rule table, model and interface. A useful small application with honest failure handling can be stronger than many disconnected features. Ask students to demonstrate one failure without making the board perform an unsafe or network-intensive test.

Suggested assessment rubric

Area	Points	Evidence
Functional application and handling of missing or unusable readings	30	Defined rule, correct update boundary, missing/zero/recovery cases
UI/UX for the specified user/task	20	Screen flow, readable information, accurate controls and feedback
Architecture and resource ownership	20	Single owners, command/event path, allocation lifetime, coherent diff
Validation and reproducibility	20	Exact configuration, meaningful checks, evidence categories and limits

Area	Points	Evidence
Explanation and handover	10	Clear report, limitations, peer reproducibility and oral source trace

Adapt weights to the course. Do not award physical-operation credit for a compiler log. If equipment or RF access is unavailable, grade the corresponding analysis and explicit evidence limit rather than encouraging unsupported success claims.

Common misconceptions to address

- Arduino `loop()` can coexist with worker tasks; it is not the entire runtime.
- A queue copies its item, but a pointer item still refers to a separately owned allocation.
- A finite zero and a missing measurement have different meanings.
- More decimal places do not imply greater sensor accuracy.
- WiFi access does not replace a LoRaWAN gateway.
- Class A is a receive-behavior class;1.1.0 is a protocol version.
- Local TX completion does not establish gateway/network delivery.
- Uptime minutes are not UTC timestamps.
- A real library in a host harness is not ESP32 emulation or physical verification.
- A protocol field range is not an electrical rating for a breakout.

Instructor handover

Keep the kit record, dependency/patch record, teacher-checked files, assigned credential procedure, RF test plan and observed limitations with the course material. When the baseline changes, update demonstrations and answer notes against the actual source. Preserve the encoder/decoder agreement and repeat relevant checks before students inherit the new version.